

Projet informatique SE3 TPI

Julien Forget, Xavier Redon et Thomas Vantroys

1 Présentation du jeu : les envahisseurs de l'espace

Vous allez programmer un très classique jeu des envahisseurs de l'espace (Space Invader). Le terrain est en deux dimensions. Les envahisseurs arrivent par le haut de l'écran en rangées qui se déplacent horizontalement, initialement de gauche à droite. Quand les rangées arrivent en butée sur un bord du terrain, elles descendent d'un cran, puis repartent dans le sens horizontal inverse. Les envahisseurs laissent tomber, à des intervalles de temps aléatoires, des bombes, qui se déplacent verticalement.

Le joueur contrôle un vaisseau pouvant se déplacer horizontalement en bas du terrain. Des boucliers situés un peu plus haut que le vaisseau permettent de le protéger des bombes. Le vaisseau peut envoyer des missiles. Plusieurs bombes et plusieurs missiles peuvent être présents sur le terrain simultanément. Un envahisseur peut être détruit par un missile. Un missile ne peut toucher qu'un envahisseur. Si le vaisseau entre en collision avec une bombe ou avec un envahisseur, il explose et la partie est terminée. La partie est gagnée lorsque tous les envahisseurs sont éliminés.

2 Organisation

2.1 Structuration du dépôt du jeu

Si vous ne l'avez pas déjà fait, créez un dépôt TPI sur votre gitlab. Ajoutez-y les répertoires suivants, ainsi qu'un système de **Makefile** pour compiler l'ensemble du projet :

- Un répertoire principal **Envahisseurs** ;
- Un sous-répertoire **Lutins** pour stocker les images représentant les différentes entités affichées ;
- Un sous-répertoire **Graphique-2.0** pour stocker la bibliothèque graphique ;
- Un sous-répertoire **ListeChaine**.

La librairie graphique et les images sont à récupérer dans `~jforget/Cours/tpi`, répertoires **Graphique-2.0** et **Lutins**.

2.2 Structures de données

Pour représenter les différents éléments à afficher à l'écran (envahisseur, vaisseau, bombe, missile, ...) vous allez créer une structure d'*Entite*. Chaque entité est caractérisée par sa position et par le lutin (sprite) la représentant à l'écran.

Le nombre d'envahisseurs, bombes, et missiles, varie au fil du jeu. Un missile tiré après un autre peut être amené à disparaître avant le premier. Les bombes les plus récemment lancées peuvent aussi devoir être effacées avant d'autres lancées plus anciennement. Enfin, les envahisseurs peuvent être détruits dans un ordre aléatoire. Nous allons donc stocker ces objets dans des listes chaînées.

Pour le reste, vous êtes libre de choisir les représentations à utiliser.

3 Fonctionnalités

Cette section donne une vision globale des fonctionnalités à réaliser. Un plan de travail est détaillé plus loin.

3.1 Gestion des bombes

La fonctionnalité de gestion des bombes doit mettre en œuvre le comportement suivant :

- Larguer des bombes depuis un envahisseur, à intervalle de temps aléatoire ;
- Gérer le déplacement vertical des bombes ;
- Les bombes sortant par le bas du terrain sont supprimées de la liste ;
- Les bombes touchant un bouclier sont supprimées de la liste.

3.2 Gestion des missiles

La fonctionnalité de gestion des missiles doit mettre en œuvre le comportement suivant :

- Permettre de tirer des missiles depuis le vaisseau ;
- Gérer le déplacement vertical des missiles ;
- Les missiles sortant par le haut du terrain sont supprimés de la liste ;
- Une collision avec un envahisseur entraîne la suppression du missile et de l'envahisseur.

3.3 Gestion des envahisseurs

La fonctionnalité de gestion des envahisseurs doit mettre en œuvre le comportement suivant :

- Initialiser plusieurs rangées d'envahisseurs au lancement de la partie ;
- Déplacer les envahisseurs, horizontalement et verticalement.

3.4 Moteur du jeu

Le moteur de jeu pour une partie donnée exécute les étapes suivantes :

- Initialisation : envahisseurs, vaisseau, boucliers, ... ;
- Boucle sur les actions suivantes :
 - Déplacement : calcul des nouvelles positions des envahisseurs, bombes, missiles ;
 - Contrôle du vaisseau : déplacement, tir ;
 - Collisions : test sur les coordonnées et suppression si collision bombe/vaisseau, envahisseur/vaisseau, missile/envahisseur, ...
 - Test fin de jeu : vérifier si la partie est terminée ;
 - Rafraîchissement : redessiner toutes les entités à l'écran ;
 - Attendre un peu, pour gérer la vitesse de rafraîchissement.

4 Plan de travail

Le plan de travail ci-dessous détaille les étapes de réalisation du jeu. Nous recommandons vivement de respecter l'ordre indiqué. Les éléments *Bonus* ne sont à traiter que si vous considérez que vous êtes en avance sur le plan de travail.

1. git, make, affichage, refresh, temporisation
 - Afficher un monstre
 - Attendre
 - Effacer le monstre
 - En afficher un autre
 2. Structures de données
 - Structure **entite** pour les objets affichables
 - Structure **listeEntites**
 3. Initialisation du jeu
 - Une ligne de monstres
 - *Bonus* : plusieurs lignes de monstres, plusieurs types de monstres
 - Un vaisseau
 - *Bonus* : boucliers
- (8h)
4. Déplacement entités
 - Monstres : gauche vers droite
 - Monstres : changement de sens arrivé au bord
 - Lacher des bombes
 - *Bonus* : changement de lutin au cours du déplacement
 - *Bonus* : seuls les monstres du bas lachent des bombes
 5. Contrôle vaisseau
 - Quitter
 - Droite/gauche, un déplacement par appui. *Bonus* : déplacement jusqu'à relâchement de touche

- Tirer, déplacer missiles. *Bonus* : limiter la cadence de tir
- (16h)
6. Calcul de collisions
 - Fonction de collision entre deux entités
 - Fonction de collision entité/liste d'entités
 - Collision vaisseau/envahisseur, vaisseau/bombes
 - *Bonus* : Fonction de collision liste/liste
 - *Bonus* : collision envahisseur/missiles, missiles/bombes, boucliers
 - *Bonus* : changement de lutin sur collision
 - *Bonus* : résistance sur le vaisseau/sur les monstres
 7. Fin de jeu
 - Gagner/perdre
 - Stopper le jeu
 - *Bonus* : afficher un message texte (gagné/perdu)
 - *Bonus* : niveaux de difficulté, plusieurs niveaux
- (22h)